

Trabajo Práctico 1: Pruebas del software - Uso de un framework de testing

Institución: Universidad Nacional de Tucumán (UNT) - Facultad de Ciencias Exactas y Tecnología (FACET)

Cátedra: Ingeniería de Software 1 (PU) / Ingeniería de Software 2 (LI / II)

Alumnos:

- Sebastian Enrique Zelarayan
- Sosa Joaquin Humberto
- Figueroa Gonzalo Nicolas

Tecnologías: C# (.NET), xUnit y Moq

0. Código de Producción (Lógica de Negocio)

A continuación se presenta la implementación final de las clases Producto y Tienda, utilizando interfaces para habilitar el uso de dobles de prueba y aplicando el manejo de excepciones requerido.

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Linq;
```

```
namespace TiendaApp
```

```
{
```

```
    public class Producto
```

```
    {
```

```
        public string Nombre { get; set; }
```

```
        public decimal Precio { get; set; }
```

```
        public string Categoria { get; set; }
```

```
        public Producto(string nombre, decimal precio, string categoria)
```

```
        {
```

```
            Nombre = nombre;
```

```
            Precio = precio;
```

```
            Categoria = categoria;
```

```
        }
```

```
        // Marcamos el método como 'virtual' para que Moq pueda sobrescribirlo
```

```
        public virtual void ActualizarPrecio(decimal nuevoPrecio)
```

```
        {
```

```
            if (nuevoPrecio < 0)
```

```

    {
        throw new ArgumentException("El precio no puede ser negativo");
    }
    Precio = nuevoPrecio;
}
}

```

```

public class Tienda
{
    public List<Producto> Inventario { get; private set; }

    public Tienda()
    {
        Inventario = new List<Producto>();
    }

    public void AgregarProducto(Producto producto)
    {
        Inventario.Add(producto);
    }

    public Producto BuscarProducto(string nombre)
    {
        var producto = Inventario.FirstOrDefault(p => p.Nombre == nombre);

        if (producto == null)
        {
            throw new KeyNotFoundException("Producto no encontrado");
        }
        return producto;
    }

    public bool EliminarProducto(string nombre)
    {
        var producto = BuscarProducto(nombre);
        Inventario.Remove(producto);
        return true;
    }

    public void AplicarDescuento(string nombre, decimal porcentaje)
    {
        var producto = BuscarProducto(nombre);
        decimal descuento = producto.Precio * (porcentaje / 100m);
        decimal nuevoPrecio = producto.Precio - descuento;
    }
}

```

```

        producto.ActualizarPrecio(nuevoPrecio);
    }

    public decimal CalcularTotalCarrito(List<string> listaNombres)
    {
        decimal total = 0;
        foreach (var nombre in listaNombres)
        {
            var producto = BuscarProducto(nombre);
            total += producto.Precio;
        }
        return total;
    }
}

```

1. Configuración Inicial y Pruebas Básicas

1.1 Implementación de Pruebas

```

using Xunit;
using TiendaApp;

public class TiendaTestsBasicos
{
    [Fact]
    public void Test_AgregarProducto()
    {
        var tienda = new Tienda();
        var prod = new Producto("Monitor", 200m, "Hardware");

        tienda.AgregarProducto(prod);

        Assert.Contains(prod, tienda.Inventario);
    }
}

```

1.2 Respuestas conceptuales

- **¿Puedes identificar pruebas de unidad y de integración en la práctica que se realizó?**

Probar métodos internos de la clase Producto de forma aislada constituye una prueba de unidad estricta. Al probar la clase Tienda instanciando objetos reales de Producto, se realiza una prueba de integración (específicamente *integración social* o *sociable unit test*), ya que el flujo de ejecución atraviesa la lógica de múltiples componentes

reales simultáneamente.

2. Pruebas con Excepciones

2.1 Implementación de Pruebas

```
using System;
using System.Collections.Generic;
using Xunit;
using TiendaApp;

public class TiendaTestsExcepciones
{
    [Fact]
    public void Test_BuscarProducto_NoExistente_LanzaExcepcion()
    {
        var tienda = new Tienda();
        var ex = Assert.Throws<KeyNotFoundException>(() =>
        tienda.BuscarProducto("Auriculares"));
        Assert.Equal("Producto no encontrado", ex.Message);
    }

    [Fact]
    public void Test_EliminarProducto_Inexistente_LanzaExcepcion()
    {
        var tienda = new Tienda();
        Assert.Throws<KeyNotFoundException>(() =>
        tienda.EliminarProducto("Proyector"));
    }

    [Fact]
    public void Test_ActualizarPrecio_Negativo_LanzaExcepcion()
    {
        var prod = new Producto("Tablet", 300m, "Electrónica");
        var ex = Assert.Throws<ArgumentException>(() => prod.ActualizarPrecio(-50m));
        Assert.Equal("El precio no puede ser negativo", ex.Message);
    }
}
```

2.2 Respuestas conceptuales

- **¿Podría haber escrito las pruebas primero antes de modificar el código de la aplicación?**
Sí. Esta práctica se denomina Test-Driven Development (TDD).
- **Descripción del proceso:**
Se ejecuta en tres fases (Red-Green-Refactor). Primero, se escribe la prueba

invocando el método y esperando la excepción (la prueba falla porque la validación no existe aún). Segundo, se escribe el código de producción mínimo (el condicional if y el throw) para que la prueba pase. Finalmente, se refactoriza el código resultante para asegurar calidad sin alterar el comportamiento.

3. Uso de dobles para aislar unidades bajo prueba

3.1 Implementación de Pruebas

```
using Moq;
using Xunit;
using TiendaApp;

public class TiendaTestsDobles
{
    [Fact]
    public void Test_AplicarDescuento_ConMock()
    {
        var tienda = new Tienda();

        // 1. Creamos el 'Test Double' (Mock) usando la librería Moq
        var mockProducto = new Mock<Producto>("Celular", 1000m, "Electrónica");

        // 2. Agregamos el objeto falso a la tienda
        tienda.AgregarProducto(mockProducto.Object);

        // 3. Ejecutamos el método
        tienda.AplicarDescuento("Celular", 20m);

        // 4. Verificamos que Tienda llamó a ActualizarPrecio pasándole 800m exactamente 1 vez
        mockProducto.Verify(p => p.ActualizarPrecio(800m), Times.Once);
    }
}
```

3.2 Respuestas conceptuales

- **¿Puedes identificar 'Controladores' y 'Resguardos'?**

El *Controlador (Driver)* es el propio entorno de pruebas de xUnit que invoca a la clase y orquesta el test. El *Resguardo (Stub/Mock)* es el objeto simulado generado mediante la librería Moq, el cual provee un comportamiento predefinido y permite verificar las interacciones.

- **¿Qué es un “test double”? Otros nombres:**

Es un objeto genérico que sustituye a una dependencia real para aislar la unidad bajo prueba. Dependiendo de su función específica, se clasifican en: *Stubs* (devuelven datos predefinidos), *Mocks* (verifican interacciones), *Spies* (registran el estado interno), *Fakes* (tienen lógica funcional pero simplificada) y *Dummies* (objetos vacíos para cumplir con firmas).

4. Uso de Fixtures para Reutilización de Datos

4.1 Implementación de Pruebas

```
using System;
using Xunit;
using TiendaApp;

public class TiendaTestsFixtures : IDisposable
{
    private readonly Tienda _tiendaBase;

    // FIXTURE (Setup): Se ejecuta antes de cada prueba
    public TiendaTestsFixtures()
    {
        _tiendaBase = new Tienda();
        _tiendaBase.AgregarProducto(new Producto("Laptop", 1000m, "Electrónica"));
        _tiendaBase.AgregarProducto(new Producto("Mouse", 50m, "Electrónica"));
    }

    // Teardown: Limpieza al finalizar cada prueba
    public void Dispose()
    {
        _tiendaBase.Inventario.Clear();
    }

    [Fact]
    public void Test_BuscarProducto_Existente()
    {
        // Usamos el fixture (la tienda que armó el constructor)
        var prod = _tiendaBase.BuscarProducto("Laptop");

        Assert.Equal("Laptop", prod.Nombre);
        Assert.Equal(1000m, prod.Precio);
    }
}
```

4.2 Respuestas conceptuales

- **¿Qué es un fixture?**

Es un conjunto de datos, estados y configuraciones iniciales que se preparan antes de ejecutar una o varias pruebas, asegurando un entorno base conocido y estable.

- **Ventajas y enfoque:**

Evita la duplicación de código en la preparación de los tests y garantiza la consistencia. Corresponde principalmente a un diseño de pruebas de *Caja Negra*, ya que se preparan datos de entrada estandarizados y se analizan las salidas sin depender de los detalles de implementación interna.

- **Setup y Teardown:**

Setup es el proceso de preparación previa (instanciar objetos, poblar listas).

Teardown es la limpieza posterior (liberar memoria, borrar datos), garantizando que el estado de un test no contamine los resultados del siguiente.

5. Pruebas de Integración y Cobertura

5.1 Implementación de Pruebas

```
using System.Collections.Generic;
using Xunit;
using TiendaApp;

public class TiendaTestsIntegracion
{
    [Fact]
    public void Test_CalcularTotalCarrito_ConDescuentoPrevio()
    {
        var tienda = new Tienda();
        tienda.AgregarProducto(new Producto("Laptop", 1000m, "Electrónica"));
        tienda.AgregarProducto(new Producto("Mouse", 50m, "Electrónica"));

        // Integramos el descuento y luego sumamos el carrito
        tienda.AplicarDescuento("Laptop", 10m); // 10% descuento -> 900

        var carrito = new List<string> { "Laptop", "Mouse" }; // 900 + 50

        decimal total = tienda.CalcularTotalCarrito(carrito);

        Assert.Equal(950m, total);
    }
}
```

5.2 Respuestas conceptuales

- **¿Realizó una prueba de cobertura completa? ¿Qué tipo de cobertura utilizó?**
Se alcanzó un alto grado de *Cobertura de Sentencias (Statement Coverage)*, ya que se ejecutaron las líneas principales del código. No obstante, no es una cobertura completa de rutas lógicas (*Branch Coverage*), dado que faltaría probar casos frontera (por ejemplo, pasar un carrito vacío a *CalcularTotalCarrito* o aplicar un descuento del 0%).
- **Situación de desarrollo para integración ascendente (Bottom-up):**
En el desarrollo de este sistema, un enfoque ascendente iniciaría codificando y testeando exhaustivamente la unidad base: *Producto*. Una vez asegurado su correcto funcionamiento ante modificaciones de precio o datos no válidos, se procede a desarrollar y testear la clase *Tienda*. La prueba integraría objetos *Producto* reales dentro de los métodos de *Tienda*. Esto permite localizar rápidamente los fallos estructurales, construyendo la fiabilidad desde los cimientos hasta la interfaz pública.